

P1105R1

Leaving no room for a lower-level language: A C++ Subset

Published Proposal, 2018-10-06

This version:

<https://wg21.link/P1105R1>

Authors:

[Ben Craig](#) (National Instruments)

[Ben Saks](#) (Saks & Associates)

Audience:

SG14, SG10, SG1, EWG, LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:

github.com/ben-craig/freestanding_proposal/blob/master/core/optional_exceptions.bs

Abstract

Making exceptions, dynamic RTTI, TLS, heap, floating point, program teardown, and blocking operations optional in freestanding mode.

Table of Contents

1 Revision History

1.1 r0 -> r1

1.1.1 SG14 telecon polls (July 11, 2018)

1.1.2 SG14 post-telecon online poll (July 11-15, 2018)

1.1.3 SG14 cppcon meeting (Sep 26, 2018)

2 Introduction

3 Value of standardization

4 Before-and-after tables

4.1 No-change, implementation defined

4.2 Well-formed

4.3 Potentially ill-formed

5 Features going optional

5.1 Exceptions

5.1.1 Why make this optional?

5.1.2 What isn't changing?

5.1.3 What am I changing (and why)?

5.1.4 Alternative designs

5.1.5 ABI impact

5.2 Parts of `<exception>` header

5.2.1 What isn't changing?

5.2.2 What am I changing?

5.2.3 Why?

5.2.4 Alternative designs

5.3 Dynamic RTTI

5.3.1 What isn't changing?

5.3.2 What am I changing?

5.3.3 Why?

5.3.4 Alternative designs

5.3.5 ABI impact

5.4 Default heap storage

5.4.1 What isn't changing?

5.4.2 What am I changing?

5.4.3 Why?

5.4.4 ABI impact

5.5 Virtual destructors

5.5.1 What isn't changing?

5.5.2 What am I changing?

5.5.3 Why?

5.5.4 How could this virtual destructor ODR-use change be implemented?

5.5.5 ABI impact

- 5.6 Thread local storage
 - 5.6.1 What am I changing?
 - 5.6.2 Why?
 - 5.6.3 ABI impact
- 5.7 Floating point
 - 5.7.1 What am I changing?
 - 5.7.2 Why?
 - 5.7.3 ABI impact
- 5.8 Program start-up and termination
 - 5.8.1 What isn't changing
 - 5.8.2 Rationalization for the status quo
 - 5.8.3 What am I changing?
 - 5.8.4 Why?
 - 5.8.5 Alternative designs
 - 5.8.6 ABI impact
- 5.9 Language mandated blocking synchronization
 - 5.9.1 What am I changing?
 - 5.9.2 Why?
 - 5.9.3 ABI impact

6 Related works in progress, and future work

- 6.1 [P0709] Zero-overhead deterministic exceptions
- 6.2 [P0784] Standard containers and `constexpr`
- 6.3 [P1073] `constexpr!` functions
- 6.4 [P1066] How to `catch` an `exception_ptr` without even `try`-ing
- 6.5 Explicit control of program startup and termination

7 Common QoI issues

- 7.1 Pure virtual functions
- 7.2 Symbol name length

8 Frequently raised arguments

- 8.1 C++ doesn't standardize subsets
- 8.2 A subset will fracture the C++ user base
- 8.3 This doesn't propose one subset, it defines many subsets
- 8.4 A dialect will be burdensome to vendors
- 8.5 Everybody wants a different subset

8.6 This doesn't agree with common style guides, like MISRA

8.7 A subset will be burdensome for the committee

9 Acknowledgments

References

Informative References

§ 1. Revision History

Polls are all in the typical Strongly in Favor/Weakly in Favor/Neutral/Weakly Against/Strongly Against format.

§ 1.1. r0 -> r1

Freestanding-only `noexcept` semantics were dropped due to low value, low SG14 support, and ABI concerns.

`throw` statements now call `std::terminate`, rather than cause UB. `std::terminate` was the approach with the least opposition.

`typeid(type)` now allowed.

Added sections discussing ABI ramifications of changes.

Changed thread-safe statics feature test macro.

Clarifying `if constexpr` comments. `catch` blocks will still help determine auto return types.

Added description of potential virtual destructor implementation.

Added frequently raised arguments section.

Added design alternative for program termination.

§ 1.1.1. SG14 telecon polls (July 11, 2018)

Minutes

Poll 1: get rid of freestanding

0/1/2/9/11

Poll 2: modify freestanding along the lines of the paper, encouragement for further work, agree with most of it

5/13/4/0/0

§ 1.1.2. SG14 post-telecon online poll (July 11-15, 2018)

[Thread](#)

Poll 3: noexcept should behave differently in environments without exceptions, along the lines of the paper
0/4/3/5/0

Poll 4: make throw UB when exceptions aren't available
0/3/6/3/1

Poll 5: make throw ill-formed when exceptions aren't available
1/3/1/5/3

Poll 6: make throw call `std::terminate` when exceptions aren't available
0/4/7/2/0

§ 1.1.3. SG14 cppcon meeting (Sep 26, 2018)

[Minutes](#)

Poll: I want to know if we're on board with a way to disable dynamic, type-based exceptions (this proposal is neutral with respect to static exceptions)
(no opposition in this room)

§ 2. Introduction

Conforming C++ toolchains are ill-suited to target kernel and embedded domains. In practice, kernel and embedded developers almost always use compiler switches that make the toolchain non-conforming. This means that conforming C++ has left room for a lower-level language: non-conforming C++. WG21 needs to decide between the lesser of several evils: formalizing a dialect, leaving room for a lower-level language, or massive breakage in real code. If we do nothing, we will have left room for a lower-level language (C, non-conforming C++). If we change hosted mode in a way to achieve the zero overhead, no lower-level language goal, we will end up needing to remove valuable features, breaking massive amounts of code. This paper proposes formalizing a dialect.

It is my intent that this be the least bad form of dialect, the proper subset. All valid freestanding libraries should be valid hosted libraries with compatible semantics.

This paper proposes making the following features optional: [exceptions](#), parts of the `<exception>` header, [dynamic RTTI](#), [default heap storage](#), [thread local storage](#), [floating point](#), the [atexit family of functions](#), [locked atomics](#), and [thread-safe static initialization](#).

In [\[P0829\]](#), I propose adding library features to freestanding mode that should work everywhere. This paper covers the removal and modification of features that don't work everywhere. There is already standards precedent in [support.signal](#) for avoiding portions of all the features that I am making optional.

There are years, if not decades of field experience using C++ subsets similar to what I am proposing ([\[OSR\]](#), [\[APPLE_KERNEL\]](#)). The workarounds and compiler switches are mostly available today. The main places where this paper innovates is in places where we can keep more features than current compiler based switches allow.

In theory, this paper would result in large scale code breaks for existing freestanding users. In practice, there are almost no existing freestanding users because the current definition is not serving the stated purpose of working "[without the benefit of an operating system](#)". Existing implementations already provide mechanisms for disabling many of the features that this paper proposes to make optional. Updating these implementations to conform to this proposal would leave existing users largely unaffected, except that they would now be using a truly compliant C++ implementation.

I believe that the embedded and kernel C++ community is better served by making features optional, rather than providing conforming, but low quality, highly unsatisfactory implementations. Missing functionality sends a clear signal to library writers, where low quality implementations provide an easier to miss message.

Note that freestanding implementations can (and should) make available all the features that are implementable on their target environment. For example, there are many embedded systems where floating point operations are desirable, but heap allocations are not. Each cluster of features will get its own feature test macro. This has the effect of making all implementations compliant that are "between" the bare minimum freestanding and the full hosted implementation.

§ 3. Value of standardization

What benefit does standardization bring to the kernel and embedded communities? Kernel and embedded developers seem to be getting work done in non-conforming C++, so why should WG21 change course?

First, I will answer those questions with another question: Why bring any proposal into the standard? Presumably the authors of those proposals could get work done without the proposal. Proposal authors are resourceful people, and can probably implement their papers in a fork of an existing compiler or standard library. Yet they go through the hassle and expense of presenting papers to WG21 anyway.

By making freestanding useful, I will be providing a target for toolchain and library authors. Library authors that wish to make their libraries as portable as possible will have a standardized lowest common

denominator to write against. Purchasers will be better able to make requests of their vendors for freestanding compliant products. Educators will be better able to teach about the requirements of kernel and embedded programming. Tool vendors can better prioritize work on conforming compiler modes, and possibly reject new, ad-hoc non-conforming modes. Users can get uniform behavior on what is currently an inconsistent set of vendor extensions.

§ 4. Before-and-after tables

§ 4.1. No-change, implementation defined

Code	Today's conforming freestanding reality	Proposed conforming freestanding behavior
<pre>//namespace scope struct Obj { Obj(); ~Obj(); }; Obj dynamic_default_init; int value_init = 42; int zero_init;</pre>	Implementation defined, depending largely on behavior of the loader and whether the normal CRT entry is used / replicated. Whether <code>Obj::Obj()</code> or <code>Obj::~~Obj()</code> are ever called is implementation defined. It is implementation defined whether <code>value_init</code> contains 42 or an indeterminate value. It is implementation defined whether <code>zero_init</code> contains 0 or an indeterminate value.	No change

§ 4.2. Well-formed

Standard says this should work	Today's reality	Proposed conforming freestanding behavior
<code>throw 0;</code>	Visual Studio 2017, /kernel error C2980: C++ exception handling is not supported with /kernel gcc 8.1, -fno-exceptions error: exception handling	Proposed option: <code>throw 0;</code> and <code>throw;</code> call <code>std::terminate()</code> if exceptions are not enabled. <code>throw;</code> currently calls

	disabled, use -fexceptions to enable clang 6.0.0, -fno-exceptions error: cannot use "throw" with exceptions disabled gcc 8.1 and clang 6.0.0, -nostdlib undefined reference to "__cxa_allocate_exception" undefined reference to "__cxa_throw" undefined reference to "typeinfo for int" Bare metal gcc 4.8 with newlib undefined reference to "__exidx_end" undefined reference to "__exidx_start" undefined reference to "_exit" undefined reference to "_sbrk" undefined reference to "_kill" undefined reference to "_getpid" undefined reference to "_write" undefined reference to "_close" undefined reference to "_fstat" undefined reference to "_isatty" undefined reference to "_lseek" undefined reference to "_read"	<code>std::terminate()</code> if executed outside of a <code>catch</code> block. Alternatives to be polled: • Ill-formed if exceptions are not enabled. • Undefined behavior if <code>throw 0;</code> is executed and exceptions are not enabled.
<code>std::bad_alloc e;</code>	Visual Studio 2017, /kernel error LNK2019: unresolved external symbol "void __cdecl operator delete(void *,unsigned __int64)" error LNK2019: unresolved external symbol __std_exception_destroy	Proposed option: Well-formed, but uncommon code. Most likely to be seen in a discarded <code>catch</code> block. Alternative to be

	gcc 8.1 and clang 6.0.0, -nostdlib undefined reference to "std::bad_alloc::~~bad_alloc()" Bare metal gcc 4.8 with newlib undefined reference to "__exidx_end" undefined reference to "__exidx_start" undefined reference to "_exit" undefined reference to "_sbrk" undefined reference to "_kill" undefined reference to "_getpid" undefined reference to "_write" undefined reference to "_close" undefined reference to "_fstat" undefined reference to "_isatty" undefined reference to "_lseek" undefined reference to "_read"	polled: Ill-formed if exceptions are not enabled.
<pre>void caller() { try {foo();} catch(const std::exception &e) { log_exception(e.what()); throw; } }</pre>	Visual Studio 2017, /kernel error C2980: C++ exception handling is not supported with /kernel gcc 8.1, -fno-exceptions error: exception handling disabled, use -fexceptions to enable clang 6.0.0, -fno-exceptions error: cannot use "throw" with exceptions disabled error: cannot use "try" with exceptions disabled gcc 8.1 and clang 6.0.0, -nostdlib	Proposed option: Well-formed. When exceptions aren't present, catch generates no code. The try block is still executed, but does no exception bookkeeping, as is common in setjmp / longjmp EH implementations. Names and syntax are still checked in catch blocks, similar to if constexpr (false). auto deduced return types still use any

	undefined reference to "__cxa_begin_catch" undefined reference to "__cxa_rethrow" undefined reference to "__cxa_end_catch" undefined reference to "_Unwind_Resume" undefined reference to "typeinfo for std::exception" undefined reference to "__cxa_begin_catch" undefined reference to "std::terminate()" undefined reference to "__gxx_personality_v0" Bare metal gcc 4.8 with newlib undefined reference to "__exidx_end" undefined reference to "__exidx_start" undefined reference to "_exit" undefined reference to "_sbrk" etc...	return statements in the catch block, so as not to cause return types to vary based on presence of exceptions.
<pre>typeid(int);</pre>	Visual Studio 2017, /kernel Works! Visual Studio 2017, /GR- Works! gcc 8.1 and clang 6.0.0, -fno-rtti error: cannot use typeid with -fno-rtti gcc 8.1 and clang 6.0.0, -nostdlib undefined reference to "typeinfo for int" Bare metal gcc 4.8 with	Proposed option: Well-formed, even if dynamic RTTI is not available.

	newlib undefined reference to "__exidx_end" undefined reference to "__exidx_start" undefined reference to "_exit" undefined reference to "_sbrk" etc...	
<pre>struct B {virtual ~B() {} }; void foo() {B b;}</pre>	Visual Studio 2017, /kernel error LNK2019: unresolved external symbol "void __cdecl operator delete(void *,unsigned)"	Proposed option: Well-formed, even if the heap is not enabled.
	gcc 8.1 and clang 6.0.0, -nostdlib undefined reference to "operator delete(void*, unsigned long)" undefined reference to "vtable for __cxxabiv1::__class_type_info"	

§ 4.3. Potentially ill-formed

Standard says this should work	Today's reality	Proposed conforming freestanding behavior
<pre>struct B {virtual void f() {} }; struct D : B {virtual void f() {} }; D *func(B *b) { return dynamic_cast<D*>(b); }</pre>	Visual Studio 2017, /kernel error C2981: the dynamic form of "dynamic_cast" is not supported with /kernel gcc 8.1, -fno-rtti error: "dynamic_cast" not permitted with -fno-rtti clang 6.0.0, -fno-rtti error: cannot use dynamic_cast	Proposed option: Ill-formed if dynamic RTTI is not enabled.

	with -fno-rtti gcc 8.1 and clang 6.0.0, -nostdlib undefined reference to "__dynamic_cast" undefined reference to "vtable for __cxxabiv1::__si_class_type_info" undefined reference to "vtable for __cxxabiv1::__class_type_info" Bare metal gcc 4.8 with newlib undefined reference to "__exidx_end" undefined reference to "__exidx_start" undefined reference to "_exit" undefined reference to "_sbrk" etc...	
<pre>#include <typeinfo> struct B {virtual void f() {}}; const bool func(B &b) { return typeid(b) == typeid(int); }</pre>	Visual Studio 2017, /kernel error C2981: the dynamic form of "typeid" is not supported with /kernel gcc 8.1, -fno-rtti error: cannot use "typeid" with - fno-rtti clang 6.0.0, -fno-rtti error: cannot use typeid with -fno- rtti gcc 8.1 and clang 6.0.0, -nostdlib undefined reference to "typeinfo for int" undefined reference to "strcmp" Bare metal gcc 4.8 with newlib undefined reference to "__exidx_end" undefined reference to "__exidx_start"	Proposed option: Ill-formed if dynamic RTTI is not enabled.

	undefined reference to "_exit" undefined reference to "_sbrk" etc...	
<pre>void f(int *i) {delete i;}</pre>	Visual Studio 2017, /kernel error LNK2019: unresolved external symbol "void __cdecl operator delete(void *)" gcc 8.1 and clang 6.0.0, -nostdlib undefined reference to "operator delete(void*, unsigned long)" Bare metal gcc 4.8 with newlib undefined reference to "_sbrk"	Proposed option: Ill-formed if the heap is not enabled and operator delete has not been provided by the user.
<pre>int foo() { thread_local int x = 0; ++x; return x; }</pre>	Visual Studio 2017, /kernel error C2949: thread_local is not supported with /kernel gcc 8.1 and clang 6.0.0, -nostdlib successfully compiles, but corrupts memory associated with thread control block	Proposed option: Ill-formed if thread-local storage is not enabled.
<pre>double doubler(double x) { return x * 2.0; }</pre>	Visual Studio 2017, /kernel successfully compiles, and corrupts user-mode floating point application state unless extra code is written to preserve the floating point state Bare metal gcc 4.8 with newlib successfully compiles, and even works, at the expense of 1052 bytes of floating point addition library code	Proposed option: Ill-formed if floating point support is not enabled.
<pre>void handler(); void foo() { atexit(handler); }</pre>	Visual Studio 2017, /kernel error LNK2019: unresolved external symbol "int atexit(void)" gcc 8.1 and clang 6.0.0, -nostdlib undefined reference to "atexit"	Proposed option: Ill-formed if dynamic initialization and tear-down

	Bare metal gcc 4.8 with newlib undefined reference to "_sbrk"	support is not enabled.
<pre> struct Obj {Obj();}; void foo() { static Obj obj; } </pre>	Visual Studio 2017, /kernel successfully compiles, but generates thread unsafe initialization for <code>obj</code> . gcc 8.1 and clang 6.0.0, -nostdlib undefined reference to "__cxa_guard_acquire" undefined reference to "__cxa_guard_release" undefined reference to "__cxa_guard_abort" undefined reference to "_Unwind_Resume" undefined reference to "__gxx_personality_v0" Bare metal gcc 4.8 with newlib undefined reference to "__exidx_end" undefined reference to "__exidx_start" undefined reference to "_exit" undefined reference to "_sbrk" etc...	Proposed option: Ill-formed if blocking synchronization support is not enabled.
<pre> struct BigData { int d[16]; }; void foo(std::atomic<BigData> &lhs, const BigData &rhs) {lhs = rhs;} </pre>	Visual Studio 2017, /kernel successfully compiles, but generates spin locks that are dangerous when shared with interrupts. gcc 8.1 and clang 6.0.0, -nostdlib undefined reference to "__atomic_store" Bare metal gcc 4.8 with newlib undefined reference to "__atomic_store"	Proposed option: Ill-formed if blocking synchronization support is not enabled.

§ 5. Features going optional

The following applies only to freestanding mode. Hosted mode will remain unchanged.

The feature macros are somewhat backwards from how the macros are normally defined. The macros are defined when the paper is adopted and the feature is missing. We can't define the macros in the past to say the features are present. Testing for the "non-feature" macros is a safer and more backwards compatible way of determining whether the following features are present.

§ 5.1. Exceptions

Feature test macro: `__cpp_freestanding_no_exceptions`. Users can check `__cpp_freestanding_no_exceptions` when they want to determine what behavior `throw` will have. The lack of the pre-existing `__cpp_exceptions` macro from [\[SD6\]](#) would not provide that information.

This section applies to "dynamic" exceptions. In other words, the exceptions we have had since C++98. [\[P0709\]](#) could add "static" exceptions. I am keeping static exceptions in mind with this design, but I'm not providing any wording against that proposal.

§ 5.1.1. Why make this optional?

Kernel and embedded environments can't universally afford exceptions. Throwing an exception requires a heap allocation on the Itanium ABI, and a large stack allocation on the Microsoft ABI, neither of which are suitable in kernel and embedded environments. Throwing an exception requires TLS ([§5.6 Thread local storage](#)) in order to propagate the number of uncaught exceptions. Windows, Linux, Mac, and FreeBSD don't allow drivers to store arbitrary TLS data, and they don't have any special handling for C++ specific TLS requirements, like the number of uncaught exceptions.

Even when exceptions aren't thrown, there is a large space cost. Table based exception costs grow roughly in proportion to the size and complexity of the program, and not in the number of `throw` sites, `catch` sites, or frames traversed in an exception `throw`. Since table based exception costs grows with program size, rather than how much it is used, it is not zero overhead. `setjmp` / `longjmp` exception size costs are similar in these regards.

See [\[P0709\]](#) for further discussion on the problems with exceptions.

§ 5.1.2. What isn't changing?

`try` and `catch` are both still allowed. Compilers should treat `catch` blocks as discarded code (i.e. an `if constexpr (false)` block). `try` and `catch` blocks are allowed so that exception neutral code can be

shared between freestanding and hosted implementations without requiring preprocessor hackery.

A rethrow without an active exception currently calls `std::terminate`.

§ 5.1.3. What am I changing (and why)?

`catch` blocks are treated similarly to an `if constexpr (false)` block. This is to allow many error handling cases to continue compiling without resorting to macros. The contents of the `catch` block are discarded, but `auto` return type deduction will still respect the types in `return` statements within `catch` blocks. The `return` type of the function should not typically depend on the presence or absence of exception support.

Evaluating a `throw` expression in an environment without exception support calls `std::terminate`. We allow the programmer to compile with a `throw` to allow exception neutral code to be shared between freestanding and hosted implementations. The `throw` should never be evaluated, since we shouldn't be able to get into a `catch` block.

We allow `throw` expressions so that programmers in environments with exceptions can catch the exception, and either translate the exception to another type of exception, rethrow the exception in a "[Lippincott function](#)", or handle the exception some other way. In these cases, we have the expectation that the code will never run in the exceptionless environment.

Implementations are encouraged to produce warnings on any `throw` expression with operands, as well as allow suppressions for informing the compiler when those `throws` are actually there for exception translation purposes.

§ 5.1.4. Alternative designs

1. `throw` UB vs. ill-formed vs. `std::terminate`

We could make some or all `throw` expressions ill-formed. The benefit is that compilers could more reliably produce diagnostics. The cost is that it would be more difficult to share exception neutral code between hosted and freestanding. We have experience with this choice in the GCC and clang world with `-fno-exceptions`.

We could make `throw` statements UB. UB likely optimizes better than `std::terminate`. Compilers would be able to remove any code that leads to the UB, reducing overall binary size. We have experience with this choice in the Visual Studio world, when exceptions are disabled.

We have library experience in libc++, libstdc++, and the Visual Studio STL with turning throws into varying forms of terminates.

2. `try` and `catch` allowed vs. ill-formed

If we made `try` and `catch` ill-formed, we would severely impact the portability of libraries across the exception and non-exception worlds. However, this is basically the status quo today, so we have experience with this pain.

If we adopt everything else in this paper, while banning `try` and `catch`, we would be able to claim that freestanding C++ is signal safe C++.

3. Only allow `catch(...)` and `throw;`

Logging exceptions and translating exceptions are less common use cases than simple `catch` and rethrow use cases. Allowing `catch(type)` takes us down the path of pulling in `std::exception`, as well as making it difficult to diagnose inappropriate `throw obj;` statements. Pulling in `std::exception` means that we must also remove the hard dependency of `virtual` destructor's on `::operator delete` (see [§5.4 Default heap storage](#)).

§ 5.1.5. ABI impact

If an exception is thrown from exception-enabled code, across exception-disabled code, the results are undefined. Structure sizes and mangled names of entities should be unaffected.

§ 5.2. Parts of `<exception>` header

Feature test macro: `__cpp_freestanding_no_exceptions`.

§ 5.2.1. What isn't changing?

The `std::exception` base class will still be available. This class (and many of its children) need to exist so that hosted exception handling code can continue to log, translate, and handle errors, all while still compiling in freestanding mode.

`std::terminate` will still be available. Various language features, most recently [contracts](#), rely on `std::terminate`. Freestanding will keep `std::terminate` rather than respecify how all those features signal unrecoverable errors.

§ 5.2.2. What am I changing?

Other than `std::exception` and `std::terminate`, nothing in the `<exception>` header will be present in environments without exception support. This means the following facilities will no longer be required:

- `terminate_handler`, `get_terminate_handler` and `set_terminate_handler`
- `uncaught_exceptions`
- `exception_ptr`, `current_exception`, `rethrow_exception`, and `make_exception_ptr`
- `bad_exception` and `nested_exception`
- `throw_with_nested` and `rethrow_if_nested`

§ 5.2.3. Why?

The terminate handler functions require synchronizing a global variable. Freestanding environments do not have a reliable way to do that (see [§5.9 Language mandated blocking synchronization](#)). The default terminate handler is typically suitable.

`uncaught_exceptions` relies on thread-local storage (see [§5.6 Thread local storage](#)). Hard coding a return value of zero would work for existing implementations, but it would close off potential future designs (see [§6.1 \[P0709\] Zero-overhead deterministic exceptions](#)).

The `exception_ptr` and `throw_with_nested` facilities require heap allocations and/or thread-local storage.

§ 5.2.4. Alternative designs

1. Omit `std::exception` and its children.

This alternative would make it so that clients could only `catch(...)` and `catch` their own client defined types. This removes the ability of those clients to log or translate exceptions. However, it would likely require less work on the implementation side, seeing as the current exception classes don't work in kernel and embedded environments.

2. Omit the entire `<exception>` header.

In addition to the issues in the above alternative, we would also need to ensure that all the other library features and core language features didn't call `std::terminate` in freestanding mode.

§ 5.3. Dynamic RTTI

Feature test macro: `__cpp_freestanding_no_dynamic_rtti`. This macro is distinct from the `__cpp_rtti` macro already defined in [\[SD6\]](#). Users cannot currently (in 2018) reliably test for the presence of dynamic RTTI with `__cpp_rtti`, so dynamic RTTI should generally assumed to be present, unless `__cpp_freestanding_no_dynamic_rtti` is present.

"Dynamic" RTTI is RTTI that requires some form of dynamic dispatch to resolve. In particular, this covers `dynamic_cast` expressions and `typeid(expr)` expressions that involve classes with virtual functions. `typeid(type)` expressions can be resolved without indirection.

§ 5.3.1. What isn't changing?

`typeid(type)` is required to be present and work. Users only pay for the `type_info` objects they use with this syntax. The `<typeinfo>` header is also required to be present.

§ 5.3.2. What am I changing?

`typeid(expr)` and `dynamic_cast` are ill-formed in environments without dynamic RTTI.

§ 5.3.3. Why?

`typeid(expr)` and `dynamic_cast` generally require vtables to point at a `type_info` object. Those `type_info` objects consume space, and are difficult to optimize away. If an instance of the class is ever created, the linker isn't able to apply trivial dead data elimination techniques to get rid of the `type_info` object, as there exists a reference to the object from the vtable. `typeid(type)` doesn't require registering the `type_info` object in the vtable, so it is fine.

The slot in the vtable itself is also a place where space is wasted.

If `typeid(expr)` and `dynamic_cast` can't be called, implementations can safely remove the `type_info` objects, saving space. Some ABIs will even permit reclaiming the vtable slot.

`typeid(expr)` can throw if used on a null pointer. Since we aren't allowing `typeid(expr)`, this isn't a concern.

§ 5.3.4. Alternative designs

We could also allow the subset of `typeid(expr)` expressions that do not require dynamic dispatch. If the static type of the expression resolves to a reference to a class with virtual functions, the program would be ill-formed. I feel that this alternative would be more brittle, and more difficult to teach.

§ 5.3.5. ABI impact

The current major implementations all provide ways to disable RTTI, so there is real world experience here.

An object created in an RTTI-enabled implementation can be passed to a no-RTTI implementation, and the RTTI implementation can use it without any ill-effects. A no-RTTI implementation can create an object, and pass it to an RTTI-enabled implementation, and everything will work fine, so long as `typeid` and `dynamic_cast` are not used on the object.

A no-RTTI class can inherit from an RTTI class with no ill-effects. An RTTI-enabled class cannot universally inherit from a no-RTTI class.

`typeid(type)` across RTTI boundaries can cause trouble on some ABIs. The Microsoft ABI eagerly emits `type_info` objects and falls back to string comparisons for `type_info` objects, so it gets by just fine. The Itanium ABI doesn't always emit `type_info` objects in each TU, so some cross RTTI-boundary cases will result in missing symbols.

§ 5.4. Default heap storage

Feature test macro: `__cpp_freestanding_no_default_heap`.

§ 5.4.1. What isn't changing?

Non-allocating placement `::operator new` and `::operator delete` will still be present. Users will still be allowed to implement the replaceable allocation and deallocation functions, as well as provide class specific implementations of `operator new` and `operator delete`.

§ 5.4.2. What am I changing?

On systems without default heap storage, neither the replaceable allocation functions nor the replaceable deallocation functions are provided by default.

§ 5.4.3. Why?

Many embedded systems do not have a heap. Such a system could provide an implementation of `::operator new` that immediately throws `bad_alloc`, but that would require pulling in all the exception handling machinery. Returning `nullptr` would not be conforming, and would also take up a non-zero amount of space.

Many kernel systems have multiple pools of memory, none of which is suitable as a default. In the Microsoft Windows kernel, developers have the choice of paged pool, which is plentiful and dangerous; and non-paged pool, which is safe and scarce. The National Instruments codebase has had experience using each

of those options as a default, and both have proven problematic. The Microsoft Visual Studio compiler switch `/kernel` already implements the lack of default allocation functions. [\[kernel_switch\]](#)

§ 5.4.4. ABI impact

No subtle affects. If a program is expecting a heap to be present where it is not, then the program won't work.

§ 5.5. Virtual destructors

Feature test macro: `__cpp_freestanding_no_default_heap`.

§ 5.5.1. What isn't changing?

`virtual` destructors are still permitted and well formed.

§ 5.5.2. What am I changing?

On systems without default heap storage, the presence of a `virtual` destructor shall not require `::operator delete` to be provided unless an instance of the object is created with `new`. Constructors and destructors will not ODR-use non-placement allocation and deallocation functions. Instead `new` and `delete` expressions will ODR-use the non-placement allocation and deallocation functions. ([basic.def.odr](#))

§ 5.5.3. Why?

In current implementations of `virtual` destructors, the class's vtable points at a stub function that calls the "real" destructor, then calls `::operator delete`. This places a burden on freestanding users of hosted code, even when the freestanding users aren't using `new` and `delete`. It seems reasonable to allow a freestanding class to have a `virtual` destructor, so long as the class is never `newed` or `deleteed`. Hosted uses of the class can `new` and `delete` all they want.

§ 5.5.4. How could this virtual destructor ODR-use change be implemented?

First, this is only a problem that needs to be solved on systems without a default heap. This means that typical user-mode desktop and server implementations would be unaffected.

Existing linkers already have the ability to take multiple identical virtual table implementations and pick one for use in the final binary. A potential implementation strategy is for compilers and linkers to support a new "weaker" linkage. When the default heap is disabled, the compiler would emit a vtable with a `nullptr` or pure virtual function in the virtual destructor slot. When `new` is called, a "stronger" linkage vtable would be emitted that has the deleting destructor in the virtual destructor slot. The linker would then select a vtable with the strongest linkage available. Today's linkage would be considered "stronger". Only partially filled vtables would have "weaker" linkage.

§ 5.5.5. ABI impact

Mixing multiple object files into the same program should be fine, even if some of them have a default heap and some don't. All the regular / "strong" linkage vtables should be identical, and all the "weaker" linkage vtables should be identical. If anyone in the program calls any form of `new`, the deleting destructor will be present and in the right slot. If no-one calls `new` in the program, then no-one should be calling `delete`, and the empty vtable slot won't be a problem.

Shared libraries are trickier. Vtables aren't always emitted into every translation unit. Take shared library "leaf" that has a default heap. It depends upon shared library "root" that does not have a default heap. If a class with a virtual destructor is defined in "root", along with its "key function", then a call to `new` on the class in "leaf" will generate an object with a partial vtable. Calling `delete` on that object will cause UB (usually crashes).

Lack of a default heap should generally be considered a trait of the platform. Mixing this configuration shouldn't be a common occurrence.

§ 5.6. Thread local storage

Feature test macro: `__cpp_freestanding_no_thread_local_storage`.

§ 5.6.1. What am I changing?

Programs using the `thread_local` storage class specifier are ill-formed if the environment does not provide thread local storage.

§ 5.6.2. Why?

Thread local storage requires cooperation from the operating system.

For embedded platforms, there may not be an operating system. Implementing thread local storage on those platforms would be extra runtime overhead.

For kernel platforms, and drivers in particular, the operating system may be owned by a third party. The third party may not provide arbitrary thread local storage for plugins. Neither Linux, Microsoft Windows, Apple OSX, FreeBSD, nor OpenRTOS support arbitrary thread local storage in the kernel.

§ 5.6.3. ABI impact

Disabling TLS should have no direct effect on the ABI. In programs with mixed TLS settings, the no-TLS code should have no effect on the allocation or use of TLS in the TLS-enabled parts of the code.

If a TLS-enabled function communicates information to callers or callees via TLS, then TLS-disabled code would not be capable of using that communication channel.

§ 5.7. Floating point

Feature test macro: `__cpp_freestanding_no_floating_point_support`.

§ 5.7.1. What am I changing?

The `float`, `double`, and `long double` types are ill-formed if the environment does not have floating point support.

`<cfloat>` is not required to be present in environments without floating point support.

`numeric_limits<floating point type>` is not required to be present in environments without floating point support.

§ 5.7.2. Why?

Many embedded processors do not have floating point units. The cost for the first usage of floating point is very high, as that pulls in floating point emulation libraries.

In kernel environments, floating point operations are avoided. The system call interface from user mode to kernel mode normally does a partial context switch, where it saves off the old values of registers, so that they can be restored when returning to user mode. In order to make user / kernel transitions fast, operating systems usually don't automatically save or restore the floating point state. This means that carelessly using floating point in the kernel ends up corrupting the user mode program's floating point state.

§ 5.7.3. ABI impact

Floating point functions would not be usable (obviously). When no floating point is used, the difference between hard-float and soft-float ABIs on ARM and MIPS should be none.

Environments where floating point support is prohibited may need to use different implementations of some common functions. For example, in many environments, `memcpy` will use vectorized instructions that touch floating point registers. In an environment where floating point is prohibited (like many OS kernels), the implementation of `memcpy` will need to avoid using floating point registers.

§ 5.8. Program start-up and termination

Feature test macros:

- `__cpp_freestanding_no_static_initialization.`
- `__cpp_freestanding_no_dynamic_initialization.`
- `__cpp_freestanding_no_termination.`

§ 5.8.1. What isn't changing

`basic.start.main` already makes start-up and termination implementation defined for freestanding implementations. I interpret this as meaning that neither static initialization nor dynamic initialization is required to take place. This also means that non-local object destruction is implementation defined.

`std::abort` and `std::terminate` will remain in the library. `_Exit` will be in the library assuming [\[P0829\]](#) is accepted.

§ 5.8.2. Rationalization for the status quo

Zero-overhead is a very sharp edge. Initializing global, mutable data to zero requires the runtime code to know a range of bytes, and then the runtime code needs to memset those bytes to zero. Applications that do not care about zero initialization could have better uses for those bytes and startup time.

All code which runs before the user's code could be considered unwanted overhead in some applications. All code that runs after the user's code could also be considered unwanted overhead. Also, the "early" code that does initialization needs to be written in some language, and if we require zero initialization to happen before anything else, then that excludes C++ from being used to write early startup code.

In practice, I expect zero initialization and static initialization to be the most used freestanding extension.

`std::abort` and `_Exit` do not call global destructors, global registration functions, or flush file I/O. `std::terminate` does not call destructors or flush file I/O, but it does call a global registration function. §5.2 Parts of `<exception>` header makes the getters and setters for the global registration function optional, so a freestanding `std::terminate` doesn't necessarily have a registration function either. That leaves these as three functions that will end the program in an implementation defined way.

§ 5.8.3. What am I changing?

The existence of `atexit`, `at_quick_exit`, `exit`, and `quick_exit` should be implementation defined (i.e. optional).

§ 5.8.4. Why?

These functions require space overhead, and are difficult to optimize away. Process termination code iterates over the contents of the `atexit` list, pinning the memory in place.

§ 5.8.5. Alternative designs

We could also remove `std::abort`, `std::terminate`, and `_Exit`. The C library implementers don't necessarily know how to exit the program in whatever random environment a customer uses. On some environments, jumping to address zero is a legitimate way to reset the processor. In other environments, a system call is more appropriate.

If we removed these functions, we would have no way to signal any kind of fatal error. We would need to evaluate what that means for `throw` statements, contracts, and other language facilities.

We could potentially use technology similar to replaceable `::operator new` and `::operator delete`. Hosted implementations could provide a replaceable default terminate handler that could be replaced by a user-provided default terminate handler at build time. Freestanding implementations would not provide a default implementation. Usages of facilities that call `std::terminate` would be ill formed if a default terminate handler were not provided.

§ 5.8.6. ABI impact

None?

Mixing TUs with different startup / termination settings may cause confusion ("Why do some globals get constructed but not others?"), but I do not foresee any ABI problems.

§ 5.9. Language mandated blocking synchronization

Feature test macros:

- `__cpp_freestanding_no_locked_atomics`.
- `__cpp_freestanding_no_non_global_dynamic_static_init`. This implies that `__cpp_threadsafe_static_init` is undefined.

§ 5.9.1. What am I changing?

In environments without blocking synchronization support, dynamic initialization of function statics and non-lock-free atomics are ill-formed.

In practice, this won't require changes from toolchain vendors. On unknown environments, the C++ runtime functions necessary to implement locked atomics and dynamic initialization of function statics generally aren't provided. This results in linker errors, satisfying the ill-formed requirement. This change will make such a toolchain conforming.

This change would break code migrating from C++98 to C++Next, as it will remove function static initialization that previously worked. That same code would likely break in the C++98 to C++11 migration, as the function static initialization would require facilities not present in the environment. Implementations would likely continue to provide compiler flags to aid the migration.

§ 5.9.2. Why?

Blocking is hard and not universally portable.

On a system without an OS, your main blocking choices are disabling interrupts and spin locks. Spin locks are needed to synchronize among multiple hardware threads, and disabling interrupts is required when synchronizing a processor with itself. Neither blocking technique is universally applicable, even when limited to the realm of OS-less systems.

In the Windows kernel, there are multiple types of locks. No one lock type is appropriate in all situations.

The CRECT RTOS [\[CRECT\]](#) doesn't have independent locks like many other OSes do. All locks are explicitly associated with a particular resource. Jobs must list all resources they use so that scheduling priorities can be calculated at compile-time. This effectively means that a CRECT application has N distinct lock types, used only by that application. None of these locks are known to the maintainers of CRECT, and none of them are known to the C++ runtime. Current compiler ABIs do not provide the C++ runtime with information about the type or address of the function static being initialized.

Some OSes and applications are trying to meet hard real time guarantees. Spin locks and disabled interrupts can add potentially unbounded jitter and latency to time critical operations, even when the operation isn't performed on a time critical code path.

Some OSes aren't scheduled in a time-sliced manner. Spin locks on these systems are a bad idea. You could get in the middle of static initialization, get an interrupt that causes you to change threads, then get stuck on the initialization of the same static. Forward progress will be halted until another interrupt happens at some indeterminate point in the future.

All of these concerns are also concerns with regards to signals. [support.signal](#) already calls out that locked atomics result in UB when invoked from a signal. Dynamic initialization of a static variable is also UB when invoked from a signal. If we are willing to make special rules for signals, shouldn't we be willing to make special rules for embedded and kernel... especially if the rules are largely the same?

§ 5.9.3. ABI impact

None?

If this paper had reverted thread-safe statics back to thread-unsafe statics, then there would be typical ODR problems. Making the functions ill-formed avoids that problem though.

§ 6. Related works in progress, and future work

§ 6.1. [\[P0709\]](#) Zero-overhead deterministic exceptions

Static exceptions have the potential to be suitable for freestanding environments. The usage of TLS for [uncaught_exceptions](#) is currently the main sticking point, but a potential option there is for a freestanding implementation to not track the number of in-flight exceptions.

Efforts were made to not design out static exceptions. If we were to ignore static exceptions and other potential implementations of exceptions, we could provide an implementation of [uncaught_exceptions](#) that always returned 0. This would enable [scope_success](#) and [scope_failure](#) out of [\[P0052\]](#).

§ 6.2. [\[P0784\]](#) Standard containers and [constexpr](#)

In theory, any program (including kernel and embedded program) should be able to use [constexpr](#) containers. However, the proposal for [constexpr](#) containers requires [std::allocator](#). Kernel and embedded systems may not want to provide [std::allocator](#) at runtime. There aren't general purpose ways of providing [constexpr](#) classes at compile time without also providing them at runtime. If this paper progresses, we may need to find a general purpose way of providing things at compile time, or we may need

to find a special purpose way that will satisfy the `std::allocator` use case. Note that if we only solve the special case, we will likely need to solve other special cases, like `std::vector`.

One possible avenue for the `std::allocator` special case is for the implementation to provide declarations of all the methods, but provide no implementations. The declarations may prove sufficient for the `constexpr` use case, while triggering linker errors in the runtime case.

Or maybe, this could be tackled with conditionally `constexpr!` functions...

§ 6.3. [P1073] `constexpr!` functions

P1073 provides a way to force a function to only be invocable at compile time. Freestanding implementations could mark all `constexpr`, non-freestanding functions as `constexpr!`.

§ 6.4. [P1066] How to `catch` an `exception_ptr` without even `try-ing`

P1066 makes it possible to use `exception_ptr` without `try`, `catch`, or `throw`. This may mean that it would be usable in an environment with no exceptions. The feature would still require RTTI and the heap.

§ 6.5. Explicit control of program startup and termination

At some point in the future, I would like to see a standard way to explicitly invoke constructors of globals and class statics, and a way to explicitly invoke the termination code. This would give freestanding users the ability to control when these actions take place.

§ 7. Common QoI issues

§ 7.1. Pure virtual functions

In freestanding environments, compilers should prefer to fill in vtable slots for pure virtual functions with a null pointer, rather than with a pointer to a library support function (e.g. `__cxa_pure_virtual`). The library support function takes up a small amount of space, all to support ease of debugging.

§ 7.2. Symbol name length

Some systems (including certain configurations of the Linux kernel) keep around symbol names during runtime. C++ symbol names usually encode return type information, parameter type information, enclosing namespaces and class names, and template arguments. All this extra information makes for long, and often

cryptic symbol names. The long symbol names take up more space in the resulting binary, and the mangling scheme makes for more difficult debugging.

The C++ standard does not govern name mangling, and this paper makes no concrete recommendations. Implementations should strive to allow users to make useful tradeoffs between symbol name length, legibility, and ABI compatibility.

§ 8. Frequently raised arguments

§ 8.1. C++ doesn't standardize subsets

C++ already has subsets and optional features. C++98 through C++17 have a freestanding subset, that claims to be for systems without an OS. This paper (and [\[P0829\]](#)) are trying to fulfill that goal.

[support.signal](#) identifies a subset of the language that is usable in signal handlers.

`constexpr` expressions are a subset of the language and library, with the additional burden of syntax.

Most of `<cstdint>` is optional. `random_device` is optionally useful. `native_handle` functions and `native_handle_type` typedefs are optional.

§ 8.2. A subset will fracture the C++ user base

The user base is already fractured, and has been for 20+ years. This paper may allow more exception neutral code to be shared between code bases, even with the code bases have different exception settings. I doubt we will ever be able to eliminate the user base fracture, but with the right feature additions and guidance, we can attempt to reduce the magnitude of the fracture.

§ 8.3. This doesn't propose one subset, it defines many subsets

No and yes.

This paper proposes one lowest common denominator implementation. Users can rely on that lowest common denominator to exist. This is the one subset the paper defines.

Vendors are allowed to provide hosted features that aren't in the minimal freestanding subset. This paper provides feature test macros and scoping of the feature test macros. This paper doesn't go into depth about the interaction of the various hosted features. This paper hints at these many subsets. None of the subsets that are between freestanding and hosted can be relied upon in a standards compliant portable manner. If a

user relies on a freestanding implementation that also provides dynamic RTTI (for example), then there is no guarantee that another vendor will provide a subset with the same set of features.

§ 8.4. A dialect will be burdensome to vendors

This proposal increases the amount of available work, but does not increase the amount of required work.

Vendors are only required to provide one conforming mode. If a vendor doesn't have a user base that is interested in freestanding, they won't be forced to provide it in order to be conforming. Conversely, if a vendor's clients only care about freestanding, those vendors won't be forced to provide a hosted implementation in order to be conforming.

This proposal does expand the amount of potential work for vendors. A vendor could choose to try and provide a full hosted toolchain, a minimal freestanding toolchain, and many different sets of features in between hosted and minimal freestanding. Which subsets to implement is left to the discretion of the vendor.

§ 8.5. Everybody wants a different subset

This is mostly true. That's why this paper aims for the lowest common denominator, and leaves extensions to the lowest common denominator up to vendors.

§ 8.6. This doesn't agree with common style guides, like MISRA

This paper concerns itself with what is technologically possible. Style guides are too domain specific for something as general as a lowest common denominator subset.

§ 8.7. A subset will be burdensome for the committee

Maintaining freestanding will require a small amount of ongoing committee time.

On the language front, we should be striving for freestanding compatible in the first place. Language features that are not freestanding compatible are likely not zero-cost abstractions.

On the library front, we should watch for library features that naturally lend themselves to being freestanding, and encourage the authors to design their library as such. Some library features don't naturally lend themselves to being freestanding though, and that's fine. I/O facilities and dynamically sized containers are examples of facilities that can safely ignore freestanding mode.

My expectation is that we spend more time worrying about ABI compatibility than we do about what is in freestanding and what isn't.

§ 9. Acknowledgments

Thank you to the many reviewers of this paper: Brandon Streiff, Irwan Djajadi, Joshua Cannon, Brad Keryan, Alfred Bratterud, and Phil Hindman

§ References

§ Informative References

[APPLE_KERNEL]

Apple Documentation. [The libkern C++ Runtime](https://developer.apple.com/library/archive/documentation/DeviceDrivers/Conceptual/WritingDeviceDriver/CPlusPlusRuntime/CPlusPlusRuntime.html). URL: <https://developer.apple.com/library/archive/documentation/DeviceDrivers/Conceptual/WritingDeviceDriver/CPlusPlusRuntime/CPlusPlusRuntime.html>

[CRECT]

Emil Fresk; Odin Holmes; Carlos van Rooijen. [crect: A C++, compile-time, reactive RTOS](https://github.com/korken89/crect). URL: <https://github.com/korken89/crect>

[KERNEL_SWITCH]

Microsoft Documentation. [/kernel \(Create Kernel Mode Binary\)](https://docs.microsoft.com/en-us/cpp/build/reference/kernel-create-kernel-mode-binary). URL: <https://docs.microsoft.com/en-us/cpp/build/reference/kernel-create-kernel-mode-binary>

[OSR]

The NT Insider. [Global Relief Effort - C++ Runtime Support for the NT DDK](http://www.osronline.com/article.cfm?article=57). URL: <http://www.osronline.com/article.cfm?article=57>

[P0052]

Peter Sommerlad; Andrew L. Sandoval. [Generic Scope Guard and RAII Wrapper for the Standard Library](http://wg21.link/P0052). URL: <http://wg21.link/P0052>

[P0709]

Herb Sutter. [Zero-overhead deterministic exceptions: Throw values](http://wg21.link/P0709). URL: <http://wg21.link/P0709>

[P0784]

Louis Dionne; et al. [Standard containers and constexpr](http://wg21.link/P0784). URL: <http://wg21.link/P0784>

[P0829]

Ben Craig. [Freestanding Proposal](http://wg21.link/P0829). URL: <http://wg21.link/P0829>

[P1066]

Mathias Stearn. [How to `catch` an `exception\\_ptr` without even `try`-ing](http://wg21.link/P1066). URL: <http://wg21.link/P1066>

[P1073]

Richard Smith; Andrew Sutton; Daveed Vandevoorde. [constexpr! functions](#). URL: <http://wg21.link/P1073>

[SD6]

Clark Nelson. [SD-6: SG10 Feature Test Recommendations](#). URL: <http://wg21.link/sd6>